

CS259 Mini-Project 1 — Part 2: Single-Head Attention CUDA Kernels

1. Hardware and Theoretical Bounds

GPU: NVIDIA TITAN V (Volta GV100)

Parameter	Value
FP32 peak compute	14,900 GFLOPS (14.9 TFLOPS)
HBM2 peak bandwidth	652.8 GB/s
L2 cache	4.5 MB
Shared memory / SM	96 KB
SMs	80
Ridge point	22.82 FLOPs/byte

2. Parallelization Strategy (Q1)

Prefill kernels

Each query row i is independent, so the natural mapping is **one CUDA block per query row**: `Grid = (S,)`. Within a block, $D=64$ threads handle the D output dimensions in parallel — one thread owns one output element and accumulates its result across all attended keys.

For the flash kernels, the K/V tile (16 or 32 rows) is loaded cooperatively by all 64 threads into shared memory; each thread contributes `tile_rows` load operations strided by D . The cross-thread dot-product reduction uses `__shfl_down_sync` within each warp and a two-element `swarp[]` array for cross-warp communication.

Scalability:

- Context length S :** Grid grows linearly with S , which is ideal. However, flash attention must re-read K and V for every query row; total DRAM traffic grows as $O(S^2)$, so the kernel transitions from compute-bound ($S=4096$) to memory-bound ($S=65536$) as S increases.
- Batch size:** Not parallelized — the current implementation is single-batch. Adding a batch dimension would require an additional grid axis `Grid = (S, batch)` and proportionally more memory.

- **Head dimension D:** Hard-coded as 64 (one thread per dimension). Changing D would require restructuring the block and the warp-level reduction.

Decode kernels

Decode has only one query vector; the natural parallelism is across the C context positions. The flash decode kernels use **one block per slice of C**: `Grid = ([C/slice],)`, `Block = D=64` threads. This distributes K/V reads across multiple SMs.

Scalability:

- **Context length C:** More blocks launch as C grows, improving SM utilization. For C=4096 (16 blocks), most SMs are idle; for C=65536 (256 blocks), all 80 SMs are engaged but at only ~10% occupancy — the problem is still too small to saturate the GPU.
- **Batch size:** Same limitation as prefill — not parallelized across batch.
- **Slice size:** Smaller slices launch more blocks (better SM coverage) but increase Phase 2 merge cost. Slice=256 was chosen to balance these effects.

3. Algorithmic FLOPs (Q2)

Prefill (causal, D=64)

Each output row i attends to keys $j = 0, 1, \dots, i$ (causal masking). The computation for row i is:

1. **QK dot products:** $i+1$ keys $\times$ D FMAs = $(i+1) \cdot D$ FMAs
2. **Weighted V sum:** $i+1$ values $\times$ D FMAs = $(i+1) \cdot D$ FMAs

Total FMAs per row $i = 2 \cdot (i+1) \cdot D$ (the factor of 2 counts both QK and AV passes).

Summing over all rows and converting FMAs to FLOPs (1 FMA = 2 FLOPs):

$$\begin{aligned} \text{FMAs} &= \sum_{i=0}^{S-1} 2 \cdot (i+1) \cdot D \\ &= 2 \cdot D \cdot S \cdot (S+1) / 2 \\ &= D \cdot S \cdot (S+1) \end{aligned}$$

$$\text{FLOPs} = \text{FMAs} \times 2 = 2 \cdot D \cdot S \cdot (S+1)$$

The factor of 2 in the final answer comes from the FMA→FLOPs conversion (1 FMA = 1 multiply + 1 add = 2 FLOPs), *not* from double-counting QK and AV — those are already accounted for in the per-row FMA count.

Workload	S	FLOPs
Prefill S=4096	4,096	$2 \times 64 \times 4096 \times 4097 \approx \mathbf{2.15 \text{ GFLOP}}$
Prefill S=65536	65,536	$2 \times 64 \times 65536 \times 65537 \approx \mathbf{549.8 \text{ GFLOP}}$

Decode (D=64)

One query vector attends to all C context positions:

1. **QK dot products:** C keys $\times$ D multiply-adds = C·D FLOPs
2. **Weighted V sum:** C values $\times$ D multiply-adds = C·D FLOPs

Total = **2·C·D**. The softmax itself (exp, division) is not counted as FFMA but contributes to execution time.

Workload	C	FLOPs
Decode C=4096	4,096	$2 \times 4096 \times 64 \approx \mathbf{0.52 \text{ MFLOP}}$
Decode C=65536	65,536	$2 \times 65536 \times 64 \approx \mathbf{8.39 \text{ MFLOP}}$

Note: NCU counts $4 \cdot C \cdot D$ FFMA instructions for decode (not $2 \cdot C \cdot D$) because the online softmax update — $s = s \cdot \text{corr} + es$ and $o = o \cdot \text{corr} + es \cdot sV$ — adds roughly $2 \cdot C \cdot D$ additional multiply-adds beyond the bare QK/AV math. The algorithmic FLOP count uses only the mathematically necessary operations.

4. Execution Time (Q3)

Kernel	Size	Time (ms)	GFLOPS	Speedup vs naive
prefill_naive	S=4096	7.356	292.0	1.00×
prefill_flash	S=4096	5.444	394.6	1.35×
prefill_flash_v2	S=4096	8.433	254.7	0.87×
prefill_naive	S=65536	—	—	N/A (OOM)
prefill_flash	S=65536	1406.6	390.8	—
prefill_flash_v2	S=65536	2128.0	258.4	0.66× vs flash
decode_naive	C=4096	0.417	2.52	1.00×
decode_flash	C=4096	0.109	9.66	3.83×
decode_flash_v2	C=4096	0.106	9.85	3.91×
decode_naive	C=65536	8.700	1.93	1.00×
decode_flash	C=65536	0.186	90.0	46.8×
decode_flash_v2	C=65536	0.183	91.5	47.5×

GFLOPS = algorithmic FLOPs / measured time.

5. Roofline Analysis (Q4)

Theoretical arithmetic intensity

Theoretical AI assumes each input matrix element is read exactly once and each output element is written once:

Minimum bytes (float32 = 4 bytes):

- Prefill: read Q ($S \cdot D \cdot 4$) + read K ($S \cdot D \cdot 4$) + read V ($S \cdot D \cdot 4$) + write O ($S \cdot D \cdot 4$) = $4 \cdot S \cdot D \cdot 4$ bytes
- Decode: read q ($D \cdot 4$) + read K ($C \cdot D \cdot 4$) + read V ($C \cdot D \cdot 4$) + write o ($D \cdot 4$) $\approx 2 \cdot C \cdot D \cdot 4$ bytes (q, o negligible)

Workload	Algorithmic FLOPs	Min bytes	Theoretical AI
Prefill S=4096	2.148×10^9	$4 \times 4096 \times 64 \times 4 = 4.194 \text{ MB}$	512 FLOPs/byte
Prefill S=65536	549.8×10^9	$4 \times 65536 \times 64 \times 4 = 67.1 \text{ MB}$	8,192 FLOPs/byte
Decode C=4096	0.524×10^6	$2 \times 4096 \times 64 \times 4 = 2.10 \text{ MB}$	0.25 FLOPs/byte
Decode C=65536	8.39×10^6	$2 \times 65536 \times 64 \times 4 = 33.6 \text{ MB}$	0.25 FLOPs/byte

Measured arithmetic intensity (from NCU)

Measured AI = (ffma_count $\times$ 2) / (dram_read + dram_write)

Kernel	Size	DRAM Read	DRAM Write	ffma count	Measured AI	Occupancy	DRAM BW%
prefill_naive	S=4096	33.88 MB	47.76 MB	1,149M	28.2 FLOPs/B	47.88%	1.66%
prefill_flash	S=4096	3.15 MB	1.19 MB	5,371M	2,475 FLOPs/B	29.71%	0.12%
prefill_flash_v2	S=4096	3.15 MB	1.20 MB	5,371M	2,469 FLOPs/B	14.57%	0.08%
prefill_flash	S=65536	742.72 GB	18.21 MB	1,374B	3.70 FLOPs/B	34.08%	80.90%
prefill_flash_v2	S=65536	11.56 GB	18.21 MB	1,374B	237 FLOPs/B	15.57%	0.83%
decode_naive	C=4096	2.10 MB	0.51 KB	561K	0.53 FLOPs/B	7.30%	0.60%
decode_flash	C=4096	2.10 MB	1.28 KB	2,621K	2.49 FLOPs/B	3.12%	2.64%

Kernel	Size	DRAM Read	DRAM Write	ffma count	Measured AI	Occupancy	DRAM BW%
decode_flash_v2	C=4096	2.11 MB	1.02 KB	2,621K	2.48 FLOPs/B	3.12%	2.93%
decode_naive	C=65536	33.96 MB	1.90 MB	8,978K	0.50 FLOPs/B	7.30%	0.63%
decode_flash	C=65536	33.56 MB	1.44 MB	41,943K	2.40 FLOPs/B	9.89%	38.29%
decode_flash_v2	C=65536	33.56 MB	1.44 MB	41,943K	2.40 FLOPs/B	9.96%	40.07%

Theoretical vs measured AI — what the difference reveals

Kernel	Theoretical AI	Measured AI	Ratio	Explanation
prefill_naive S=4096	512 FLOPs/B	28.2 FLOPs/B	18× lower	Writes S×S score buffer (81 MB) not in theoretical min-bytes
prefill_flash S=4096	512 FLOPs/B	2,475 FLOPs/B	4.8× higher	Online softmax adds ~4.7× extra FFMA vs bare QK+AV; data cached in L2
prefill_flash S=65536	8,190 FLOPs/B	3.70 FLOPs/B	2,200× lower	K/V re-read S/2 times each on average; total DRAM = 742 GB not 67 MB
decode_naive C=65536	0.25 FLOPs/B	0.50 FLOPs/B	2× higher	Writes score buffer (C×4 bytes) beyond minimum; slightly more FFMA from softmax
decode_flash C=65536	0.25 FLOPs/B	2.40 FLOPs/B	9.6× higher	Online softmax FFMA inflates numerator; DRAM close to minimum

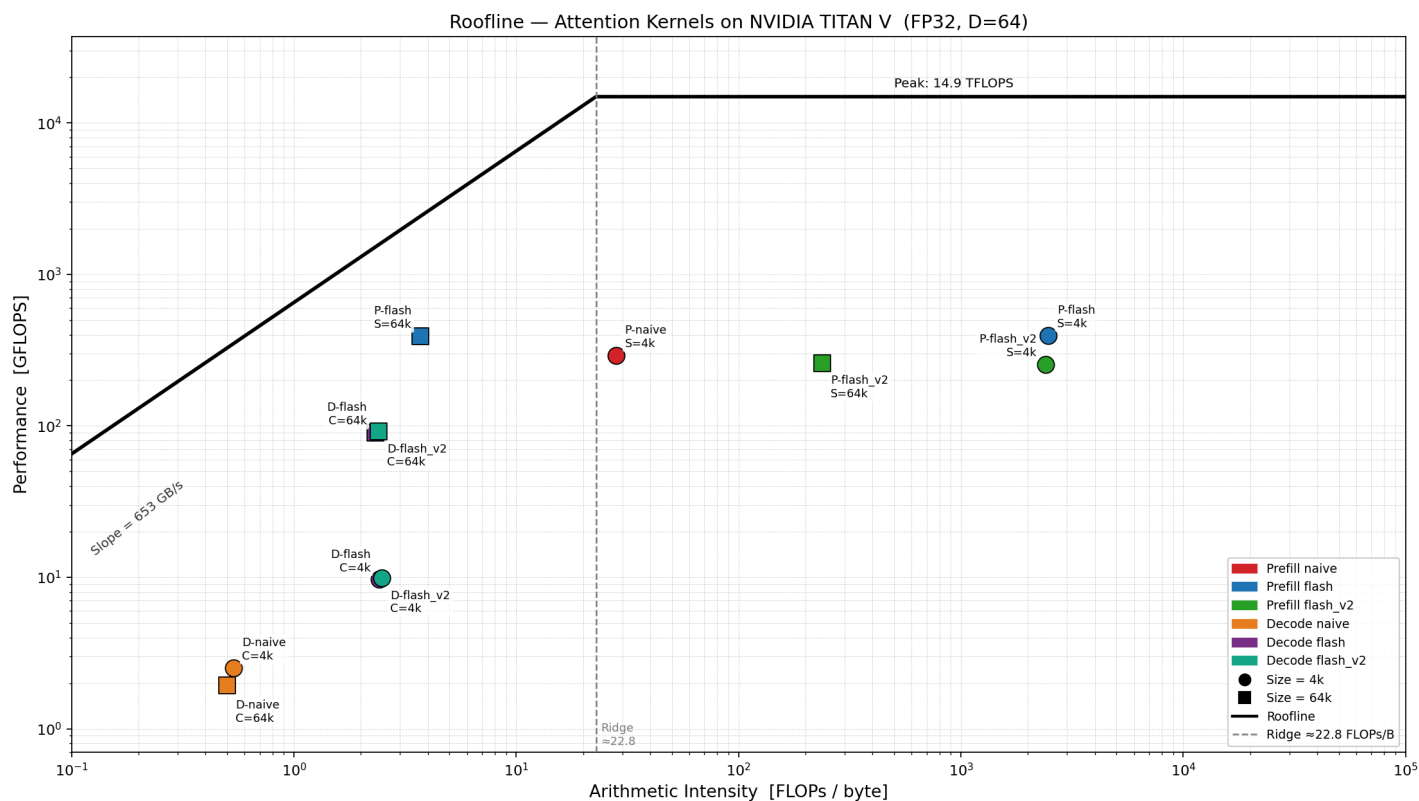
Key observations:

- **prefill_naive:** measured AI far below theoretical because the score buffer write (47.76 MB) is not in the theoretical minimum — theory assumes one read/write per element, but naive writes S^2 scores to DRAM.
- **prefill_flash S=4096:** measured AI far *above* theoretical because (a) online softmax multiplies instruction count by ~4.7× and (b) all data fits in L2, so DRAM denominator is near minimum while FFMA numerator is large.
- **prefill_flash S=65536:** the single largest discrepancy (2,200×). Theoretical AI assumes K/V are each read once (67 MB), but the causal pattern forces re-reading every key for every later query, accumulating

742 GB of DRAM traffic. This is the signature of working-set overflow from HBM.

- **decode:** measured AI is higher than theoretical mainly due to the online softmax FFMA overhead. Both naive and flash read approximately the theoretical minimum DRAM bytes.

Roofline placement and bound identification



Kernel	Measured AI	Ridge = 22.82	Bound	% of roofline ceiling
prefill_naive S=4096	28.2 FLOPs/B	> ridge	compute	$292 / 14900 = \mathbf{2.0\%}$
prefill_flash S=4096	2,475 FLOPs/B	>> ridge	compute	$395 / 14900 = \mathbf{2.7\%}$
prefill_flash_v2 S=4096	2,469 FLOPs/B	>> ridge	compute	$255 / 14900 = \mathbf{1.7\%}$
prefill_flash S=65536	3.70 FLOPs/B	< ridge	memory	$391 / (652.8 \times 3.70) = \mathbf{16.2\%}$
prefill_flash_v2 S=65536	237 FLOPs/B	>> ridge	compute	$258 / 14900 = \mathbf{1.7\%}$
decode_naive C=4096	0.53 FLOPs/B	<< ridge	memory	$2.52 / (652.8 \times 0.53) = \mathbf{0.73\%}$
decode_flash C=4096	2.49 FLOPs/B	<< ridge	memory	$9.66 / (652.8 \times 2.49) = \mathbf{0.59\%}$

Kernel	Measured AI	Ridge = 22.82	Bound	% of roofline ceiling
decode_flash_v2 C=4096	2.48 FLOPs/B	<< ridge	memory	$9.85 / (652.8 \times 2.48) =$ 0.61%
decode_naive C=65536	0.50 FLOPs/B	<< ridge	memory	$1.93 / (652.8 \times 0.50) =$ 0.59%
decode_flash C=65536	2.40 FLOPs/B	< ridge	memory	$90 / (652.8 \times 2.40) =$ 5.7%
decode_flash_v2 C=65536	2.40 FLOPs/B	< ridge	memory	$91.5 / (652.8 \times 2.40) =$ 5.8%

All kernels are far from their respective ceilings. The potential for further optimization depends on the bound:

- **Compute-bound kernels** (prefill S=4096): the bottleneck is the online softmax serial dependency chain ($m \rightarrow \text{corr} \rightarrow \text{es} \rightarrow s \rightarrow o$), which prevents instruction-level pipelining. Occupancy is also limited by shared memory. Further gains would require software pipelining or FP16/tensor-core math to increase throughput.
- **Memory-bound kernels** (prefill S=65536, all decode): the bottleneck is HBM bandwidth. For prefill at large S, the only path forward is reducing total DRAM traffic — e.g., by processing multiple query rows per block to increase K/V reuse. For decode, the problem is too small to saturate the GPU even at 256 blocks; larger batch sizes or longer context lengths would help.

6. Implementation Overview (Q5 context)

6.1 Prefill Kernels

prefill_naive (baseline, $S \leq 4096$ only)

- Grid: (S,) — one block per query row. Block: 256 threads.
- Allocates an $S \times S$ global score buffer. For S=4096 this is 67 MB; for S=65536 it would require 17.2 GB, exceeding TITAN V's 12 GB HBM — this kernel cannot be launched for large S.
- Three passes: (1) dot-products $\rightarrow$ score buffer, (2) global max + exp/sum for softmax, (3) weighted V accumulation.

prefill_flash (tile = 16)

- Implements Flash Attention with online softmax, eliminating the score buffer entirely.
- Grid: (S,), Block: D=64 threads — one thread per output dimension.
- Loads K/V in tiles of 16 rows into shared memory. Score reduction uses `__shfl_down_sync` (intra-warp) and `swarp[2]` (cross-warp). Running state (m, sum, out) updated per key using: $m_{\text{new}} = \max(m, \text{score})$; $\text{corr} = \exp(m - m_{\text{new}})$; $s = s \cdot \text{corr} + \exp(\text{score} - m_{\text{new}})$.

prefill_flash_v2 (tile = 32)

- Identical to `prefill_flash` but doubles tile size to 32 rows, halving outer-loop iterations and `__syncthreads()` barriers.
- Shared memory doubles from ~8 KB to ~16 KB per block (occupancy penalty, see Section 7).

6.2 Decode Kernels

decode_naive (baseline)

- Grid: (1,), Block: 256 threads. Global score buffer of size C.
- Three-pass softmax. Single block → single SM → 79 SMs idle.

decode_flash (2-phase Flash Decoding, slice = 256)

- Phase 1: $\lceil C/256 \rceil$ blocks, each processing 256 context positions with online softmax, writing (m, d, O_{partial}) to global.
- Phase 2: single block merges all partials using online softmax merge: $m_{\text{new}} = \max(m, m_b)$; rescale and accumulate.

decode_flash_v2 (slice = 256, float4 loads)

- Identical to `decode_flash` but replaces scalar loads with 128-bit `float4` loads, reducing load instruction count.

7. Analysis

7.1 Prefill — necessity of Flash Attention

Standard attention for $S=65536$ requires $65536^2 \times 4$ bytes = **17.2 GB** for the score matrix, exceeding TITAN V's 12 GB HBM. Flash Attention eliminates this by maintaining only $O(\text{tile} \cdot D)$ shared memory state regardless of S.

7.2 Prefill — occupancy from first principles

Occupancy is constrained by shared memory per SM (96 KB):

Kernel	smem / block	Max blocks / SM	Threads / SM	Theoretical occupancy
prefill_flash	8,208 B	$\lfloor 96 \text{ KB} / 8208 \text{ B} \rfloor = 11$	$11 \times 64 = 704$	$704 / 2048 = \mathbf{34.4\%}$
prefill_flash_v2	16,400 B	$\lfloor 96 \text{ KB} / 16400 \text{ B} \rfloor = 5$	$5 \times 64 = 320$	$320 / 2048 = \mathbf{15.6\%}$

NCU confirms: flash = 29.7%, flash_v2 = 14.6% — matching the smem-limited prediction. Doubling the tile halves occupancy.

7.3 Prefill — compute-bound vs memory-bound

S=4096 — compute-bound

Both flash kernels: measured AI $\approx$ 2,470 FLOPs/byte $\gg$ ridge (22.82). DRAM throughput $<$ 0.12%.

- `prefill_flash`: sm__throughput = 57.5%, occupancy = 29.7% $\rightarrow$ 395 GFLOPS. Bottleneck: online softmax serial dependency (m $\rightarrow$ corr $\rightarrow$ es $\rightarrow$ s $\rightarrow$ o) prevents ILP.
- `prefill_flash_v2`: occupancy = 14.6%, sm__throughput = 36.3% $\rightarrow$ 255 GFLOPS. The fewer syncthreads barriers cannot compensate for halved warp count.

S=65536 — memory-bound despite identical algorithm

`prefill_flash`: DRAM throughput = 80.9% ($\approx$ 528 GB/s), DRAM read = 742.72 GB. Measured AI = 3.70 FLOPs/byte (below ridge). Key j is read by queries j through S-1, averaging S/2 $\approx$ 32,768 re-reads; total traffic $\approx$ 1.1 TB $\gg$ 4.5 MB L2.

Why DRAM BW = 80.9% but roofline efficiency = only 16.2%: Causal load imbalance — block i processes i+1 keys. Early blocks (small i) finish almost instantly and their SMs go idle; late blocks dominate runtime. The 80.9% DRAM figure reflects only the final waves of large blocks, not the average across the full kernel. The roofline model assumes uniform load distribution, so the gap is a direct measure of the imbalance penalty.

S=65536, flash_v2 — tradeoff

DRAM read drops 64 $\times$ (11.56 GB vs 742 GB) due to better L2 reuse from the larger tile. Yet performance is worse (258 vs 391 GFLOPS): occupancy drops from 34% to 16%, sm__throughput falls from 56.8% to 37.0%. Cache efficiency gains are outweighed by occupancy loss.

7.4 Decode — multi-block parallelism

All decode kernels: AI $\approx$ 0.5–2.5 FLOPs/byte $\ll$ ridge. Memory-bound.

decode_naive: DRAM throughput = 0.63% with single block — only one SM issues memory requests, leaving 99.4% of HBM bandwidth idle.

decode_flash: 256 blocks $\rightarrow$ DRAM throughput rises to 38.3% $\rightarrow$ **46.8 $\times$ speedup**. Hundreds of concurrent memory requests better utilize HBM.

decode_flash_v2 (float4): Same 256 blocks. L1-texture traffic increases from 33.70 MB to 98.71 MB (wider vector fetches), DRAM throughput rises to 40.1% $\rightarrow$ 91.5 vs 90.0 GFLOPS.

Root cause of 40% ceiling: 256 blocks $\times$ 64 threads = 16,384 threads vs GPU capacity of 163,840. Only $\sim$ 10% occupancy — the problem is simply too small to saturate the hardware.

8. Optimizations (Q5)

Optimization	Workload	Expected benefit	Actual outcome	Root cause
Flash Attention (no score buffer)	Prefill S=65536	Enables large-S	Essential (naive OOM)	Eliminates $O(S^2)$ buffer
Flash Attention	Prefill S=4096	Reduce DRAM traffic	+35% speedup (1.35×)	DRAM traffic 81 MB → 4.3 MB
Larger tile 16→32 (flash_v2)	Prefill S=4096	Fewer syncthreads	Slower (0.87×)	smem 8→16 KB; occupancy 34%→15%; sm__throughput 57%→36%
Larger tile 16→32 (flash_v2)	Prefill S=65536	Better L2 cache reuse	Slower (0.66×)	64× less DRAM but occupancy penalty dominates
Multi-block flash decode	Decode C=65536	More SM utilization	+47× speedup	DRAM BW% 0.63%→38%; all 80 SMs engaged
float4 loads (flash_v2)	Decode C=65536	Wider memory transactions	+1.7% (marginal)	Both kernels read identical DRAM bytes; float4 marginally reduces L1 pressure
float4 loads (flash_v2)	Decode C=4096	Same as above	Negligible	Problem too small; GPU idle time dominates

Most impactful: Flash Attention for prefill (eliminates OOM for large S; 1.35× for small S) and multi-block Flash Decoding (47× speedup by utilizing all SMs).

Least impactful / negative: Larger tile (flash_v2) — consistently slower because the shared memory occupancy penalty outweighs both the barrier reduction and cache reuse benefits. This reveals that for kernels with serial dependency chains, **occupancy is the primary throughput lever**, not cache efficiency.

9. Conclusion

Flash Attention is indispensable for large prefill sequences: standard attention requires 17.2 GB for S=65536, exceeding hardware limits, while flash attention operates with $O(\text{tile-D})$ shared memory regardless of S. At S=4096, flash attention achieves a **1.35× speedup** (DRAM traffic 81.6 MB → 4.3 MB; compute-bound at 2.7% of FP32 peak). At S=65536, the bottleneck shifts to HBM bandwidth (80.9% utilization) because K/V must be re-read for each query row, accumulating 742 GB of DRAM traffic despite each matrix being only 16 MB; causal load imbalance further limits roofline efficiency to 16.2%.

Attempts to improve prefill via a larger tile (flash_v2) revealed a fundamental tradeoff: $2\times$ tile $\rightarrow 64\times$ less DRAM traffic but half the occupancy due to doubled shared memory, resulting in net slowdown. This shows that for kernels with serial dependency chains, occupancy — not cache efficiency — is the primary throughput determinant.

For decode, multi-block Flash Decoding provides a **47 $\times$ speedup** by distributing the KV cache across 256 blocks and raising DRAM throughput from 0.6% to 40%. Both tasks remain far from theoretical limits; further improvements would require breaking the online softmax dependency chain (warp-level software pipelining) or tensor-core FP16/BF16 computation.